

Описание структуры закодированного файла (метод Хаффмана)

Коба Владислав

СПбПУ, Прикладная математика и информатика

31 мая 2026 г.

Закодированный файл, формируемый разработанной программой, представляет собой бинарный файл, состоящий из двух логических частей: **заголовок (словаря)** и **блока сжатых данных**.

1. Структура заголовка (Словарь частот)

Поскольку алгоритм Хаффмана использует префиксные коды переменной длины, для успешного декодирования необходимо восстановить оригинальное дерево. Вместо сохранения самого дерева или битовых строк, в файл записывается частотный словарь.

Заголовок имеет следующую структуру:

- **Размер словаря** (4 байта, тип `int`) — количество уникальных байт (символов), встречающихся в исходном файле. Пусть это число равно N .
- **Массив частот** (размер $N \times 5$ байт) — содержит N пар данных. Для каждого уникального символа записывается:
 - Сам символ (1 байт, тип `byte`).
 - Абсолютная частота его появления в файле (4 байта, тип `int`).
- **Длина исходного файла** (4 байта, тип `int`) — точное количество байт в файле до сжатия. Этот параметр критически важен, так как бинарные данные записываются порциями по 8 бит. Если последняя порция битов не кратна 8, она дополняется незначащими нулями (`padding`). Данное поле позволяет декодеру остановиться ровно на последнем полезном байте и проигнорировать добавленные нули.

2. Структура закодированных данных

Сразу после заголовка располагается непрерывный битовый поток закодированных данных.

- Каждый символ исходного файла заменяется на битовый код Хаффмана (состоящий из битов 0 и 1), сгенерированный на основе восстановленного дерева.
- Биты накапливаются в буфере и записываются в файл строго порциями по 8 бит (1 байт).

- Последний байт при необходимости сдвигается влево (дополняется нулями) для формирования полного октета.

Пример логической компоновки файла:

```
[ int: количество_символов ] [ byte: символ_1 ] [ int: частота_1 ] ... [ int:  
длина_исходного_файла ] [ byte... : сжатые_данные ]
```

Такая архитектура позволяет безошибочно работать как со строковыми (текстовыми), так и со скомпилированными бинарными (.class) файлами.